Desarrollo back-end con PHP, framework MVC y CMS

Producto 2

Aplicación de gestión de reparaciones

Leonardo Cuevas

David Casero

Eduard Garriga

Universitat Oberta de Catalunya - Jesuïtes Educació

Abril de 2026

CONTENIDOS

MAPA CONCEPTUAL & DIAGRAMA UML/MVC **3**
PROMPTS IA **4**
PROMPT 1 4
PROMPT 2 4
PROMPT 3 4
PROMPT 4 5
HISTORIAL GIT **5**
PROCESO DE DESARROLLO **7**
BASE DE DATOS 7
REBUILD 9
FRONT CONTROLLER 10
PÁGINA HOME 11
REGISTRO Y LOGIN 12
AMPLIACIÓN Y DATA BBDD 15
PÁGINA DASHBOARD 15
PANEL PARTICULAR 15
PANEL ADMINISTRADOR 16
GESTIÓN DE INCIDENCIAS ADMIN 17
CALENDARIO 18
PANEL TÉCNICO 19
GESTIÓN DEL PERFIL 19
RESTRICCIÓN 48H DESDE SERVIDOR 19
CREACIÓN DE TÉCNICOS 20
MEJORAS DE CRUD Y ACCIONES QUE FALTAN 21
CANCELAR AVISO (CLIENTE) 21
CANCELAR Y MODIFICAR AVISO (ADMIN) 21
DESPLIEGUE A AWS 22

ENLACE GITHUB

https://github.com/egarrigame/backendphp-p2

ENLACE VÍDEO GENERAL

https://youtu.be/ZOWEqR7w5OU

ENLACE VÍDEO DAVID

https://youtu.be/YuFs5VuuNYY

ENLACE VÍDEO LEONARDO

https://youtu.be/6pYsuSMOwgo

ENLACE VÍDEO EDUARD

https://youtu.be/zrmUrC8SqBA

MAPA CONCEPTUAL & DIAGRAMA UML/MVC

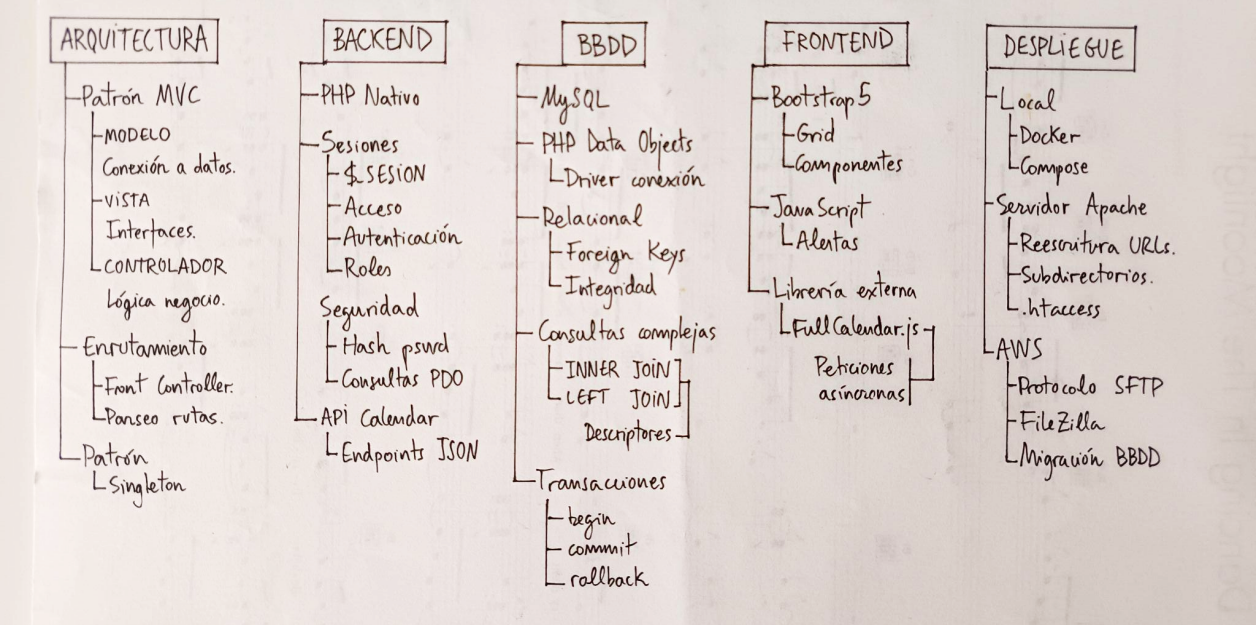
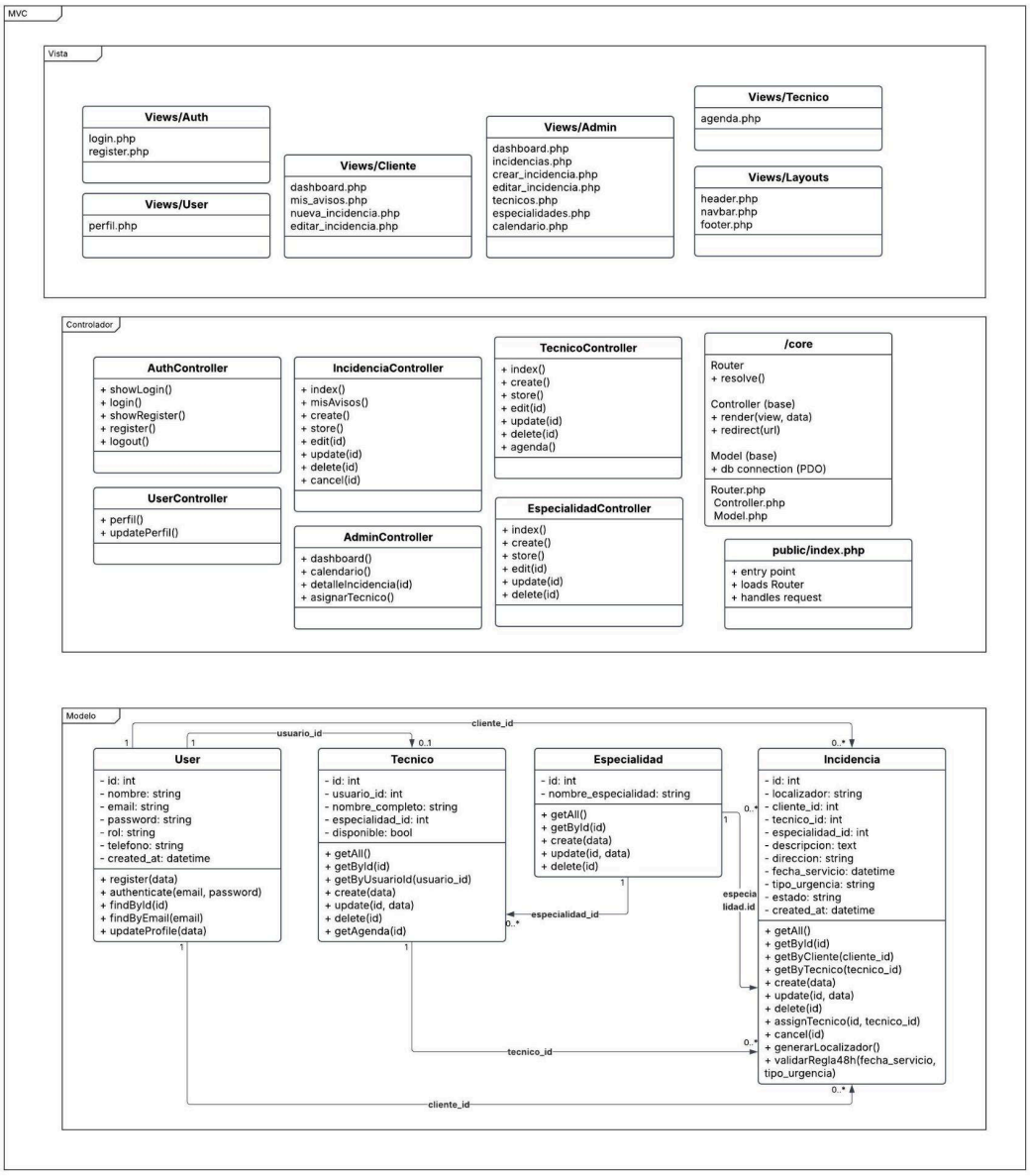


Diagrama UML – Arquitectura MVC de la aplicación.
Leonardo Cuevas. FP UOC.



PROMPTS IA

PROMPT 1

En la dirección fp064.techlab.uoc.edu/~uocx1/ sale Error 404 - Página no encontrada. La ruta /~uocx1/ no existe.

RESULTADO: Código para limpiar la URI capturada y evitar que el router interprete la carpeta como una vista.

MODIFICACIÓN: Se integró la línea *$url = str_replace('/~uocx1', '', $url);* en el archivo */Router.php* justo después de leer la petición del servidor.

EXPLICACIÓN
El router lee la URL para saber qué controlador cargar. Al desplegar la app en el servidor de AWS dentro de una subcarpeta, el router capturaba /~uocx1/ como si fuera la página que el usuario quería visitar, dando error 404. Con la función *str_replace()* de PHP borramos esa parte del texto antes de que el router la analice, y así el sistema cree que está ejecutándose en la raíz absoluta

PROMPT 2

Error de Código 500. El Router intentó cargar ../src/controllers/HomeController.php pero el archivo no existe.

RESULTADO: Identificación del problema de jerarquía de directorios y actualización de las sentencias *require_once*.

MODIFICACIÓN: Mediante la herramienta de búsqueda global, se cambian todos los *require_once '../src/'* por *require_once 'src/'* en todos los controladores.

EXPLICACIÓN
En el entorno local, el archivo */index.php* estaba dentro de una carpeta *public/*, por lo que necesitaba los dos puntos ../ para retroceder un nivel en los directorios y poder entrar a *src/*. En AWS, al mover el *index.php* al mismo nivel jerárquico que *src/*, la ruta *../* apuntaba fuera del servidor público, provocando el error 500. Quitar los puntos soluciona la ruta relativa.

PROMPT 3

En el archivo dabatase.php sale como error ya que entiendo que las variables antiguas de credenciales en local las he borrado y puesto directamente los datos de AWS. ¿Cómo deberían ser las variables de manera correcta?

RESULTADO: Refactorización del archivo de configuración para implementar el uso de constantes globales.

MODIFICACIÓN: Se reemplazaron las variables comunes (*$host, $dbname*) por la función *define()* de PHP (*define('DB_HOST', 'localhost')*).

EXPLICACIÓN
Las variables declaradas globalmente en un archivo no pueden ser leídas por defecto dentro de una función . Al usar la función *define()*, declaramos constantes globales reales que pueden ser consumidas sin problema por la instancia de PDO dentro del bloque *try-catch* de la función.

PROMPT 4

Cómo podemos hacer en cuanto a lógica que si una incidencia es de X especialidad, sólo se puedan asignar técnicos de dicha especialidad?

RESULTADO: Inserción de una validación lógica directamente en la vista para evitar asignar especialistas equivocados.

MODIFICACIÓN: En la vista de admin, se envolvió la etiqueta *<option>* dentro de un condicional *if($tec['especialidad_id'] == $inc['especialidad_id'])*.

EXPLICACIÓN
Para blindar la lógica sin cargar mucho el servidor con consultas, aprovechamos que el controlador ya envía la plantilla completa de técnicos a la vista. Usando un condicional de PHP dentro del bucle *foreach* del HTML, comparamos el ID de especialidad del técnico con el ID de la incidencia seleccionada. Si no coinciden, PHP omite la impresión de esa etiqueta *<option>*, filtrando el desplegable en tiempo real.

HISTORIAL GIT

git log --all --pretty=format:"%h | %an | %ad | %s" --date=short
845dc29 | Leonardo Cuevas | 2026-04-13 | finalizado producto 2
526bf9c | Leonardo Cuevas | 2026-04-12 | cambios finales
37153a5 | Leonardo Cuevas | 2026-04-12 | cambios finales
c0f715f | Leonardo Cuevas | 2026-04-12 | cambios en el proyecto
9357215 | Leonardo Cuevas | 2026-04-12 | cambios en el proyecto
7e919b2 | Leonardo Cuevas | 2026-04-12 | cambios proyecto
0cdce7a | Leonardo Cuevas | 2026-04-12 | cambios finales
af5ea4a | Leonardo Cuevas | 2026-04-12 | cambios en el código final
4be2689 | Leonardo Cuevas | 2026-04-11 | cambios en archivos .php
5d633af | Eduard Garriga | 2026-04-10 | final prod commit: ajustes routing servidor, subida completa
57573cf | Eduard Garriga | 2026-04-10 | feat: final cruds | desploy a AWS
7ba3699 | David Casero | 2026-04-09 | feat: agregar perfil de usuario con edicion de datos y cambio de contraseña
29c9c07 | Eduard Garriga | 2026-04-09 | feat: calendar & panel tecnico & perfil
85d75cb | Eduard Garriga | 2026-04-09 | feat: dahsboard admin & gestion incidencias
6142451 | Eduard Garriga | 2026-04-09 | feat: dashboard & form + list avisos
8406a4c | Eduard Garriga | 2026-04-09 | feat: autenticacion, registro, login y logout
c099c93 | Eduard Garriga | 2026-04-09 | feat: home page
f116b56 | Leonardo Cuevas | 2026-04-08 | Cambios en la base de datos: bbddReparaYa.sql
f03870d | Leonardo Cuevas | 2026-04-08 | conectar bbdd de ReparaYa
4957781 | Leonardo Cuevas | 2026-04-08 | Diagrama UML MVC agregado a carpeta /docs
6012ba4 | Leonardo Cuevas | 2026-04-08 | Cambios en la estructura del código, agregar BBDD
5c6609b | Eduard Garriga | 2026-04-08 | routing: htaccess & clase router done
423f71f | Eduard Garriga | 2026-04-08 | correction: eliminar servicio wp
a1f1d6a | Eduard Garriga | 2026-04-08 | initial commit: directorios & conexion Docker con bbdd
edad8d1 | David Casero | 2026-04-07 | fix: arreglar calendario y metodos del controlador base feat: implementar FontAwesome en todas las vistas del sistema
7b566fc | David Casero | 2026-04-06 | feat: CRUD incidencias clientes completo - nueva solicitud, mis avisos, cancelacion con regla 48h
aa56e15 | Leonardo Cuevas | 2026-04-02 | Create README.md
:...skipping...
845dc29 | Leonardo Cuevas | 2026-04-13 | finalizado producto 2
526bf9c | Leonardo Cuevas | 2026-04-12 | cambios finales
37153a5 | Leonardo Cuevas | 2026-04-12 | cambios finales

c0f715f | Leonardo Cuevas | 2026-04-12 | cambios en el proyecto
9357215 | Leonardo Cuevas | 2026-04-12 | cambios en el proyecto
7e919b2 | Leonardo Cuevas | 2026-04-12 | cambios proyecto
0cdce7a | Leonardo Cuevas | 2026-04-12 | cambios finales
af5ea4a | Leonardo Cuevas | 2026-04-12 | cambios en el código final
4be2689 | Leonardo Cuevas | 2026-04-11 | cambios en archivos .php
5d633af | Eduard Garriga | 2026-04-10 | final prod commit: ajustes routing servidor, subida completa
57573cf | Eduard Garriga | 2026-04-10 | feat: final cruds | desploy a AWS
7ba3699 | David Casero | 2026-04-09 | feat: agregar perfil de usuario con edicion de datos y cambio de contraseña
29c9c07 | Eduard Garriga | 2026-04-09 | feat: calendar & panel tecnico & perfil
85d75cb | Eduard Garriga | 2026-04-09 | feat: dahsboard admin & gestion incidencias
6142451 | Eduard Garriga | 2026-04-09 | feat: dashboard & form + list avisos
8406a4c | Eduard Garriga | 2026-04-09 | feat: autenticacion, registro, login y logout
c099c93 | Eduard Garriga | 2026-04-09 | feat: home page
f116b56 | Leonardo Cuevas | 2026-04-08 | Cambios en la base de datos: bbddReparaYa.sql
f03870d | Leonardo Cuevas | 2026-04-08 | conectar bbdd de ReparaYa
4957781 | Leonardo Cuevas | 2026-04-08 | Diagrama UML MVC agregado a carpeta /docs
6012ba4 | Leonardo Cuevas | 2026-04-08 | Cambios en la estructura del código, agregar BBDD
5c6609b | Eduard Garriga | 2026-04-08 | routing: htaccess & clase router done
423f71f | Eduard Garriga | 2026-04-08 | correction: eliminar servicio wp
a1f1d6a | Eduard Garriga | 2026-04-08 | initial commit: directorios & conexion Docker con bbdd
edad8d1 | David Casero | 2026-04-07 | fix: arreglar calendario y metodos del controlador base feat: implementar FontAwesome en todas las vistas del sistema
7b566fc | David Casero | 2026-04-06 | feat: CRUD incidencias clientes completo - nueva solicitud, mis avisos, cancelacion con regla 48h
aa56e15 | Leonardo Cuevas | 2026-04-02 | Create README.md
ad51840 | Leonardo Cuevas | 2026-04-02 | esqueleto inicial
a83cc2e | Leonardo Cuevas | 2026-04-02 | Esqueleto inicial
87f788e | Leonardo Cuevas | 2026-04-02 | Delete README.md
cd4bc5a | David Casero | 2026-03-29 | feat: configuracion inicial MVC y Docker funcionando
c37b09f | Eduard Garriga | 2026-03-28 | Initial commit

PROCESO DE DESARROLLO

BASE DE DATOS

Analizamos el archivo con la base de datos. Parece todo correcto, miramos ampliaciones: la tabla Especialidades está creada. En la tabla Incidencias, el campo 'estado' es un ENUM. Esto lo pasaremos a una tabla en sí, ya que es más sencillo poder hacer inserts sin tener que tocar la base de datos con modificaciones futuras.

1. Creamos tabla Estados en SQL
CREATE TABLE estados (
id INT AUTO_INCREMENT PRIMARY KEY,
nombre_estado VARCHAR(50) NOT NULL, → Pendiente, asignada, finalizada, cancelada
color_calendario VARCHAR(7) DEFAULT '#327FAD' → Para asignar color al estado

2. Modificamos ENUM de incidencias y relacionamos al estado
CREATE TABLE incidencias (
…
estado_id INT NOT NULL DEFAULT 1, → Relacionamos al primer estado
FOREIGN KEY (estado_id) REFERENCES estados(id) → Clave foránea a estados

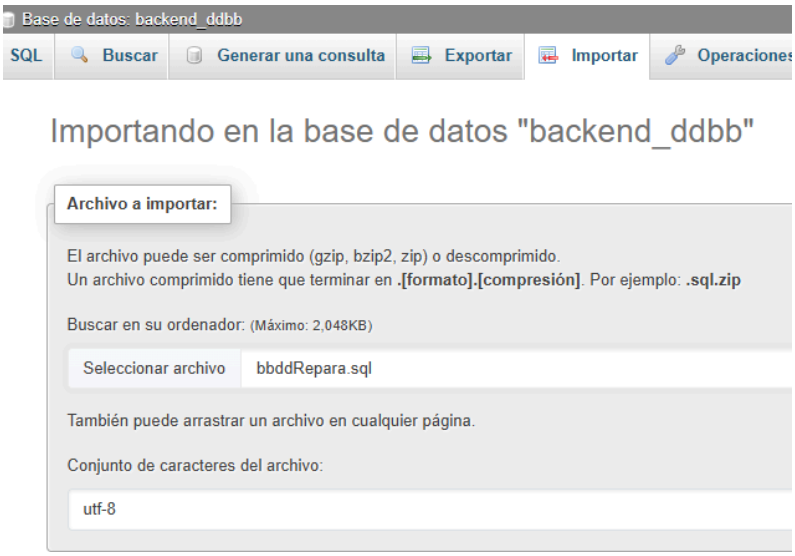
CONEXIÓN BBDD
Modificamos el archivo *src/config/database.php* que viene con código por defecto para crear nuestra configuración.

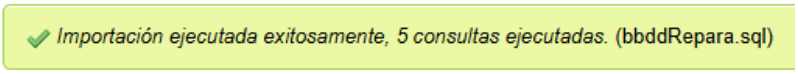
1. Ponemos la credenciales de la bbdd
define('DB_HOST', 'db'); → El servicio en el docker-compose
define('DB_NAME', 'backend_ddbb');
define('DB_USER', 'backend_user');
define('DB_PASS', 'backend_userpassword');

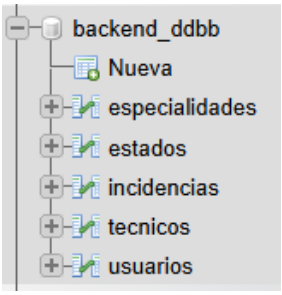
2. Creamos una función para tener la instancia de la conexión (Singleton para evitar conexiones simultáneas)
function conectarDB(): PDO {
static $pdo = null; → Variable para almacenar la conexión (singleton), la primera vez es null, después tendrá la conexión creada
if ($pdo === null) { → Si no existe la conexión, la creamos
$options → Opciones de configuración
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, → Lanzamos errores en caso de fallos
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, → Que PDO devuelva arrays asociativos (campo - valor)
PDO::ATTR_EMULATE_PREPARES => false, → Desactivar sentencias preparadas, usar nativas del motor de bbdd para mayor seguridad
$pdo = new PDO($dsn, DB_USER, DB_PASS, $options); → Instancias de PDO para conectar la bbdd
catch (PDOException $e) {
die("Error de conexión a la base de datos: " . $e->getMessage()); → Si algo falla, detenemos ejecución y mostramos el error

3. Encendemos contenedor Docker desde la terminal
docker compose up -d

4. Importamos el archivo SQL en la bbdd en phpMyAdmin



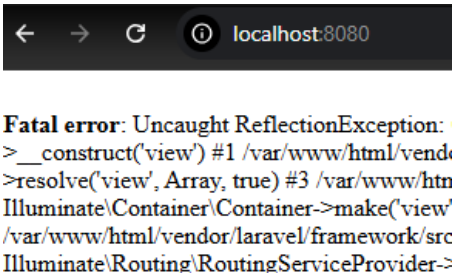




5. Creamos un archivo de prueba */index.php* en la raíz para comprobar la conexión

```
<?php
require_once 'config/database.php';
$db = conectarDB();
echo "Conexión con la bbdd desde Docker";
```

6. Accedemos al localhost:8080 para comprobar la conexión.



→ Encontramos un error fatal de ejecución. Se están intentando ejecutar Laravel. Replanteamos proyecto para empezar de 0 y no reaprovechar carpeta, que se ha hecho para poder aprovechar configuraciones y conexiones de Docker y bbdd.

REBUILD

Creamos estructura desde 0.

1. Cerramos contenedores que corren para liberar puertos desde la terminal
docker compose down

2. Creamos carpeta vacía para el proyecto y añadimos carpeta */src*

3. Copiamos el */docker-compose* y el */Dockerfile*

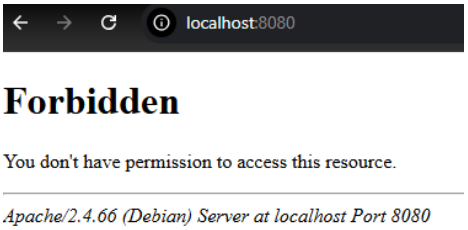
4. Creamos carpeta *src/config* y copiamos archivo */database.php*

5. Copiamos el archivo */index.php* de prueba en la raíz

6. Levantamos contenedor desde la terminal
docker compose up -d --build

7. Importamos de nuevo la bbdd en phpMyAdmin ya que es un contenedor nuevo y estaba vacía.

8. Comprobamos funcionamiento en localhost:8080



→ Apache nos deniega la entrada por la configuración de ruta única de Laravel. Replicaremos ruta para mantener separación de lógica/vista.

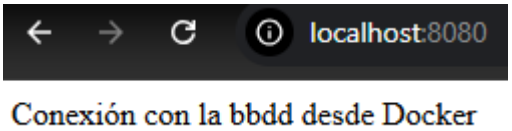
9. Creamos carpeta */public* y movemos archivo */index.php* dentro

10. Modificamos archivo */index.php* para que encuentre la nueva ruta de la configuración de la bbdd.
require_once '../src/config/database.php';

11. Modificamos ruta de Docker para que encuentre carpeta actual del servicio web
services: web: volumes:
    - ./:/var/www/html → Hemos substituido src por . para que busque en la carpeta completa y evitar error.

12. Como hemos modificado un volumen, debemos recrear el contenedor.
docker compose down
docker compose up -d

13. Comprobamos en el localhost



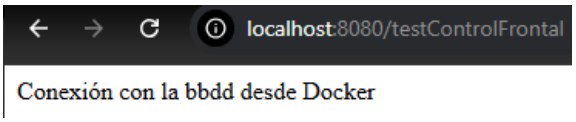
→ Conexión exitosa

FRONT CONTROLLER

En lugar de tener carpetas para todas las páginas, crearemos un hypertext access como controller frontal para que todas la URL queden limpias, le diremos a Apache que no busque carpetas y que envíe todo a /index.php, donde decidiremos qué ejecutar.

1. Creamos el archivo *public/.htaccess* para dar las instrucciones de configuración al servidor Apache
<IfModule mod_rewrite.c> → Módulo reescritura de urls
RewriteEngine On → # Encender el motor y poder manipular las url
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d → Excepciones si no son o archivos o directorios, no lo enviamos a index (imagenes...)
RewriteRule ^(.)$ index.php [QSA,L]* → Redirección a index

2. Comprobamos en el localhost



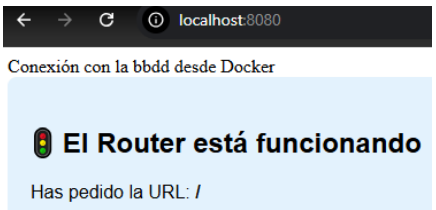
→ Una URL que no existe como página, igualmente sale mensaje del index.

Ahora creamos el router para elegir qué mostramos según la url.

1. Crear el archivo *core/Router.php*
class Router { private $rutas = [→ Array para almacenar las rutas
public function get($url, $controlador, $metodo) { → Método para registrar ruta de url
public function post($url, $controlador, $metodo) { → Método para registrar ruta de formulario
public function despachar() { → Motor del router: analizar url y cargar lo necesario
*$url = $_SERVER['RE*QUEST_URI'] ?? '/'; → Capturamos url
$url = strtok($url, '?'); → Limpiamos url
*$metodoHttp = $_SERVER['REQUEST_*METHOD']; → Comprobamos petición (GET o POST)
if (isset($this->rutas[$metodoHttp][$url])) { → Verificamos que la ruta existe
} else { http_response_code(404); → Si la url no existe, mandamos error de php

2. Modificamos */index.php* para conectar el Router y encender el enrutamiento desde el punto de entrada
require_once '../src/core/Router.php'; → Importamos la clase router que hemos creado
$router = new Router(); → Instancia del Router
$router->get('/', 'HomeController', 'index'); → Registro de rutas
$router->get('/dodo', 'PruebaController', 'metodoDodo'); → Test para ver que redirige bien
$router->get('/inventada', 'ErrorController', 'noExiste'); → Test para comprobar error
$router->despachar(); → Que Router lea la URL y ejecute lo necesario

3. Comprobamos funcionamiento router con las distintas URL



→ Gestión de rutas correcta y manejo de error, el controlador frontal y el routing funcionan.

PÁGINA HOME

CONTROLADOR HOME

Creamos el primer controlador MVC para la página de inicio de la aplicación.

1. Creamos la carpeta *src/controllers* y el archivo */HomeController.php*

class HomeController {

public function index() { → Método para llamar al router cuando alguien entre en index

$datos =

$this->render('home', $datos); → Método para buscar y cargar el html

private function render($vista, $datos = []) { → Helper para cargar el html en lugar de hacer require en cada método, le pasamos el nombre de la vista y array de datos y busca el archivo en la carpeta de vistas e inyecta las variables con la función extract

extract($datos); → Sacamos variables de los array para convertirlas en variables normales y usarlas en el html, ej: $datos['titulo'] pasa a título

$rutaVista = '../src/views/' . $vista . '.php'; → Ruta física para cargar la vista

if (file_exists($rutaVista)) { → Comprobamos que la vista existe

VISTA HOME

1. Creamos la carpeta *src/views* y el archivo */home.php*

2. Creamos una estructura y estilos con HTML y Bootstrap.

ROUTING HOME

1. Modificamos el router */Router.php* para que busque e instancie el nuevo controlador y se muestre la vista

$rutaControlador = '../src/controllers/' . $nombreControlador . '.php'; → Ruta física hasta el archivo de controlador

if (file_exists($rutaControlador)) { → Comprobamos que el archivo exista

require_once $rutaControlador; → Importamos el archivo

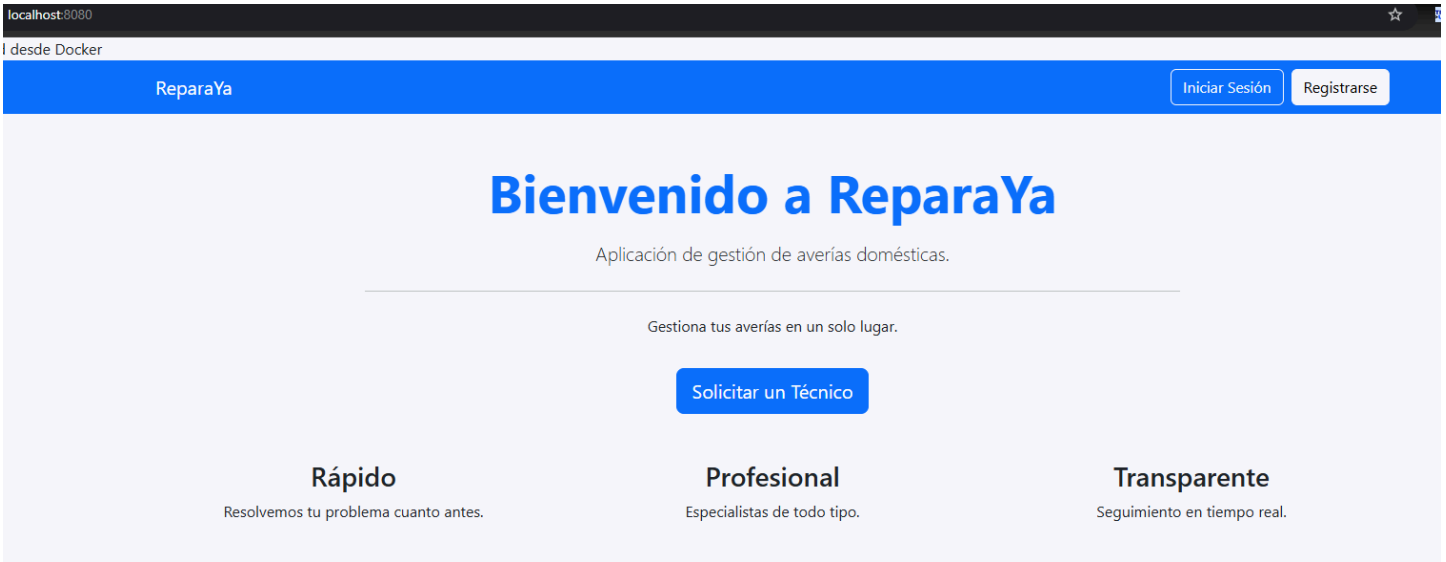
$controladorObj = new $nombreControlador(); → Instanciamos el controlador

if (method_exists($controladorObj, $nombreMetodo)) { → Comprobamos que la función exista

$controladorObj->$nombreMetodo(); → Ejecutamos el método del controlador

COMPROBACIÓN

1. Accedemos al localhost para comprobar la ruta '/'



→ Éxito en la primer página.

REGISTRO Y LOGIN

CONTROLADOR AUTENTICACIÓN

1. Creamos el archivo */AuthController.php*

public function mostrarRegistro() { → Método para mostrar el form de registro (petición GET)

private function render($vista, $datos = []) { → Método helper para cargar la vista

VISTA FORMULARIO

1. Creamos el archivo */registro.php*

2. Creamos una estructura y estilos con HTML y Bootstrap.

…

<form action="/registro" method="POST"> → Método POST para enviar información sensible como contraseña

…

<input type="text" class="form-control" id="nombre" name="nombre" required> → Atributo 'name' para que PHP pueda usar esto para guardar información en una variable

ROUTING FORMULARIO

1. Modificamos */index.php* para conectar la url con el controlador

$router->get('/registro', 'AuthController', 'mostrarRegistro');

MODELO USUARIO

1. Creamos la carpeta *src/models* y el archivo */Usuario.php*

class Usuario {

private $db; → Variable para guardar la conexión a la bbdd

public function __construct($conexionDB) { → Instanciamos la clase con la conexión a la bbdd

public function registrar($nombre, $email, $password, $telefono) { → Método para registrar un nuevo usuario

*$passwordHash = password_hash($password, PASSWORD_DEFAULT); → Bycrypt para h*ash de contraseña

$sql = "INSERT INTO usuarios (nombre, email, password, telefono, rol)

VALUES (:nombre, :email, :password, :telefono, 'particular')"; → Consulta sql con marcadores (evitar inyecciones)

$stmt->bindParam(':nombre', $nombre); → Vinculación de los datos con los marcadores de la consulta

…

return $stmt->execute(); → Ejecutamos la consulta

catch (PDOException $e) { return false; → Devolvemos false si hay error en la consulta, como correo existente o fallo de conexión

CONTROLADOR

1. Modificamos */AuthController.php* para que el controlador reciba los datos y los pase al modelo

public function procesarRegistro() { → Método para procesar los datos del formulario de registro

$nombre = $_POST['nombre'] ?? ''; → Rrecogemos los datos

if (empty($nombre) || empty($email) || empty($password)) { → Validación de campos obligatorios

require_once '../src/models/Usuario.php'; → Importamos el modelo Usuario

$db = conectarDB(); → Obtenemos la conexión a la bbdd

$usuarioModel = new Usuario($db); → Instanciamos el modelo con la conexión a la bbdd

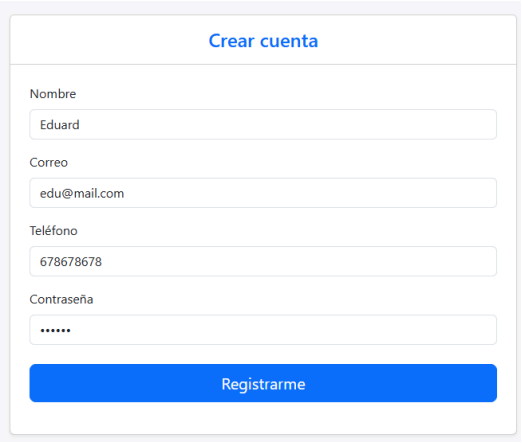
$registroExitoso = $usuarioModel->registrar($nombre, $email, $password, $telefono); → Ejecutamos el método de registrar y guardamos el resultado

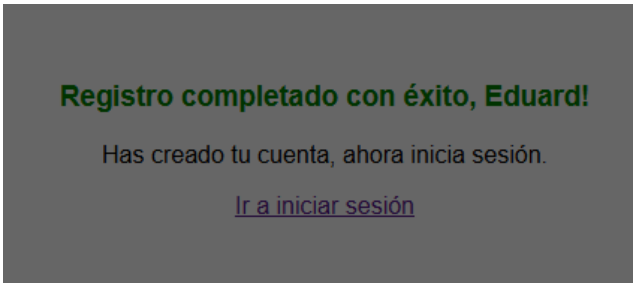
if ($registroExitoso) { → Decidimos qué mostrar

ROUTING REGISTRO

1. Modificamos */index.php* para que el router pueda procesar

$router->get('/registro', 'AuthController', 'procesarRegistro');





nombre	email	password	rol	telefono
Eduard	edu@mail.com	$2y$10$rrFu6S/YgLK2NAaB5EtkLOSCY5S45xpLtNws/RNdY1R...	particular	678678678

→ El registro funciona, ahora vamos a crear la redirección a login y el ecosistema MVC para el login.

CONTROLADOR AUTENTICACIÓN

Creamos métodos para el login

1. Modificamos */AuthController.php*

public function mostrarLogin() { → Método para el login (petición GET)

$mensajeExito = isset($_GET['exito']) ? 'Registro completado, ahora iniciar sesión.' : ''; → Recogemos la variable del registro éxito

VISTA FORMULARIO

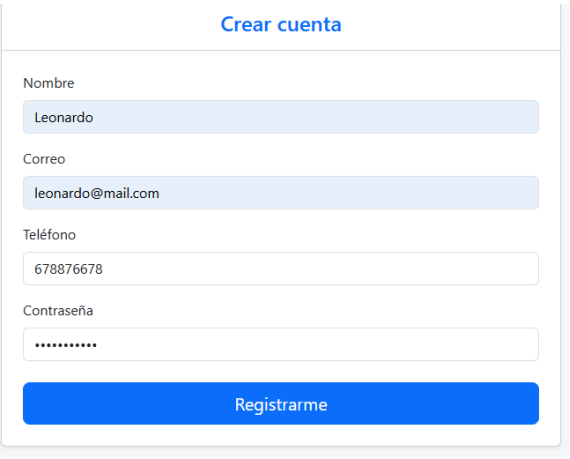
1. Creamos el archivo */login.php*

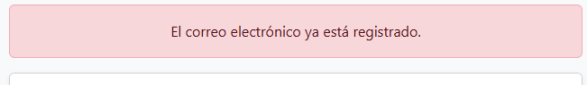
2. Creamos una estructura y estilos con HTML y Bootstrap.

ROUTING FORMULARIO

1. Modificamos */index.php* para conectar la url con el controlador

$router->get('/login', 'AuthController', 'mostrarLogin');;





→ El formulario funciona, se registra en la bbdd y redirige a login con mensaje de éxito y manejo de error al dar de alta con el mismo correo.

AUTENTICACIÓN LOGIN

1. Modificamos el modelo */Usuario.php* para verificar credenciales

public function login($email, $password) { → Método para verificar credenciales

*$sql = "SELECT * FROM usuarios WHERE email = :email";* → Buscamos correo en la bbdd

$usuario = $stmt->fetch(PDO::FETCH_ASSOC); → Nos devuelve array con los datos si lo encuentra

if ($usuario && password_verify($password, $usuario['password'])) { → Si existe, verificamos contraseña

return $usuario; → Devolvemos el usuario

SESIONES

1. Modificamos */index.php* para encender motor de sesión y añdimos ruta para procesar login

```
session_start();
$router->post('/login', 'AuthController', 'procesarLogin');
```

CONTROLADOR ACCESO

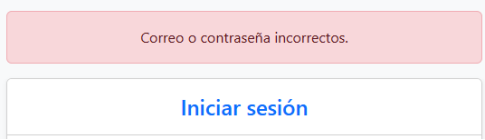
1. Modificamos */AuthController.php*

public function procesarLogin() { → Método para el login (petición POST)

require_once '../src/models/Usuario.php'; → Llamamos al modelo

$usuarioLogueado = $usuarioModel->login($email, $password); → Comprobamos credenciales

if ($usuarioLogueado) { → Éxito, guardamos sesión



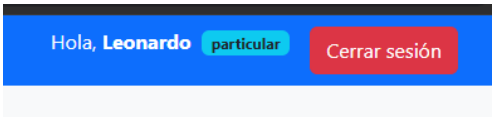
→ Gestión de error correcta al iniciar sesión con otra contraseña, y acceso correcto al redirigir a home. Registro y login exitosos.

CERRAR SESIÓN

Cuando se hace login, se deben borrar los botones Iniciar sesión y Registrarse y mostrar el de Cerrar sesión y la información del usuario

1. Modificamos la vista */home.php* para mostrar botones en función de la sesión

```
<?php if (isset($_SESSION['usuario_id'])): ?>
<a href="/logout" class="btn btn-danger">Cerrar sesión</a>
<?php else: ?>
<a href="/login" class="btn btn-outline-light me-2">Iniciar sesión</a> // <a…>Registrarse</a>
```



2. Modificamos el controlador */AuthController.php* para hacer el logout

public function cerrarSesion() { → Método para cerrar sesión

$_SESSION = []; → Vaciamos variable de sesión

session_destroy();

3. Registramos la ruta en */index.php*

$router->get('/logout', 'AuthController', 'cerrarSesion');

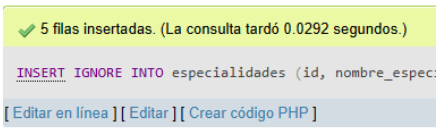
→ El sistema de logout funciona correctamente y redirige a la home con botones Iniciar sesión y Registrarse

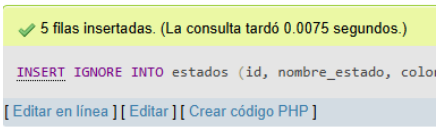
AMPLIACIÓN Y DATA BBDD

Añadimos especialidades y estados a la BBDD para tener opciones de elección.

INSERT IGNORE INTO especialidades (id, nombre_especialidad) VALUES
(1, 'Fontanería'),
(2, 'Electricidad'),
(3, 'Carpintería'),
(4, 'Electrodomésticos'),
(5, 'Cerrajería');

INSERT IGNORE INTO estados (id, nombre_estado, color_calendario) VALUES
(1, 'Pendiente', '#EDDE5C'),
(2, 'Asignada', '#5AA9E7'),
(3, 'En proceso', '#E9B16A'),
(4, 'Finalizada', '#8DBC8F'),
(5, 'Cancelada', '#DE655C');





PÁGINA DASHBOARD

PANEL PARTICULAR

MODELO INCIDENCIA

1. Creamos el modelo */Incidencia.php* para tener la estructura de los datos para el panel

CONTROLADOR DASHBOARD

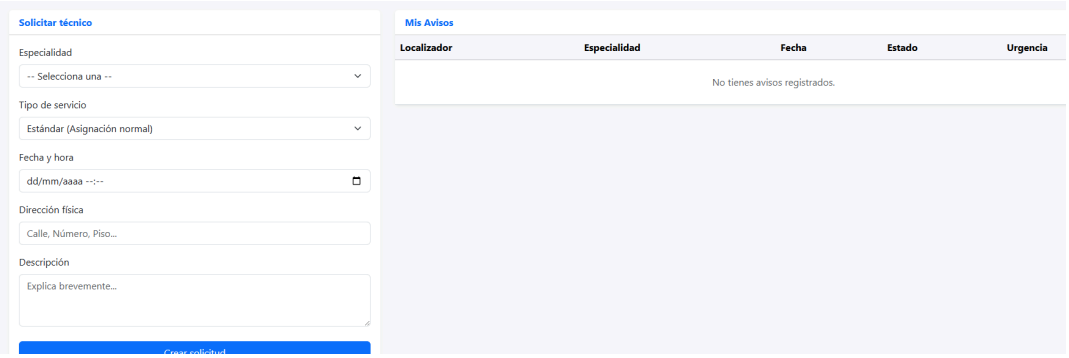
1. Creamos el archivo */DashboardController.php*

2. Registramos la ruta en */index.php*

3. Cambiamos redirección del login en */AuthController.php* para que lleve al dashboard cuando se entra
procesarLogin()
header('Location: /panel'); → Redirigimos a dashboard

4. Modificamos */DashboardController.php* para que recoja los datos del modelo y los envíe a la vista, trabajamos con el rol Particular ahora.

5. Creamos la vista */panel_cliente.php*



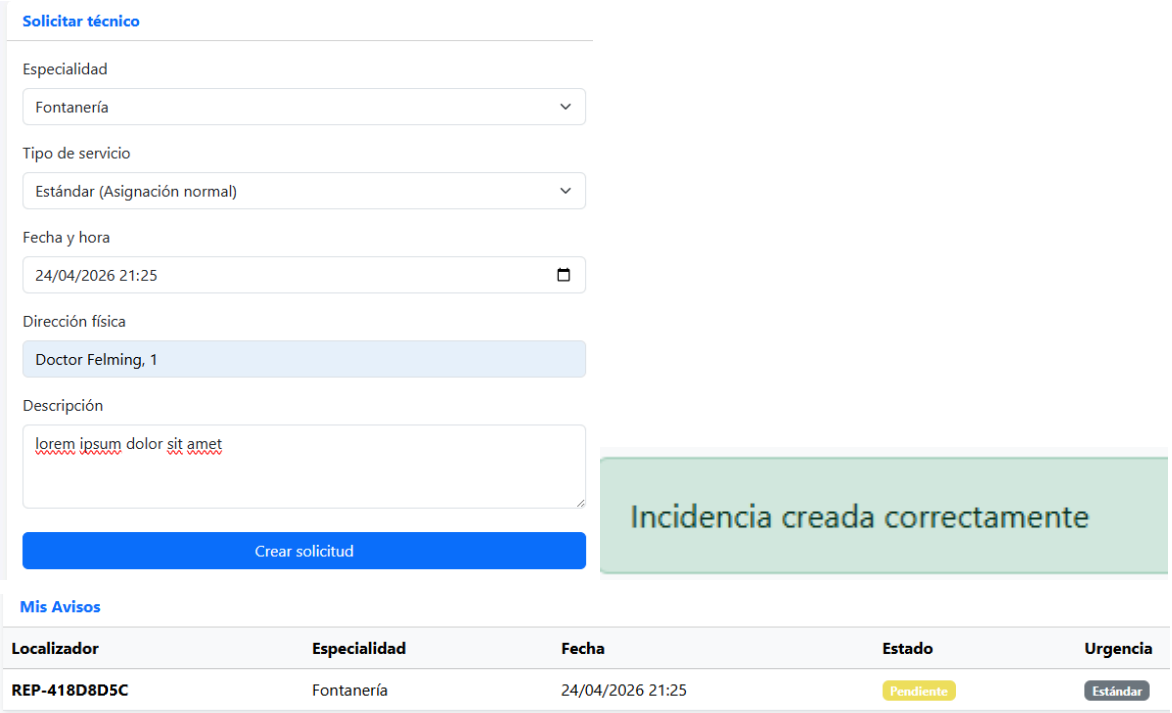
→ Form y panel para Cliente.

<u>CONTROLADOR FORMULARIO</u>

1. Modificamos modelo */Incidencia.php* para añadir método para poder crear la incidencia

2. Creamos el controlador */IncidenciaController.php*

3. Registramos la ruta en */index.php*

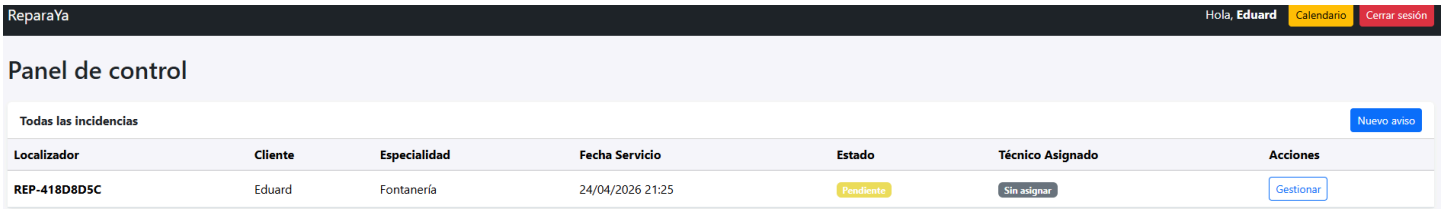


PANEL ADMINISTRADOR

1. Modificamos modelo */Incidencia.php* para añadir método para poder listar todas las incidencias

2. Actualizamos */DashboardController.php* para que muestre todas las incidencias para admins

3. Creamos la vista */panel_admin.php*



→ Listado de incidencias para admin con cliente, técnico y opción para gestionar.

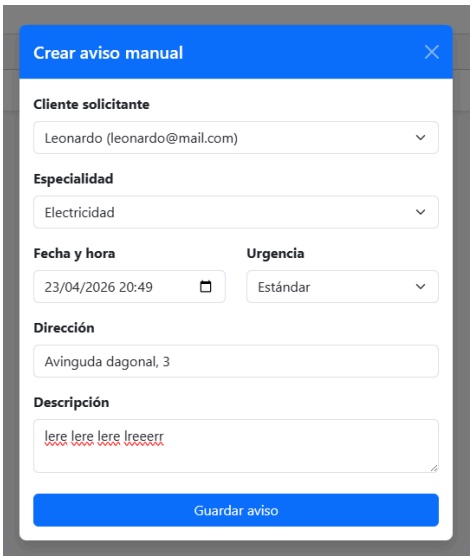
AVISOS MANUALES ADMIN

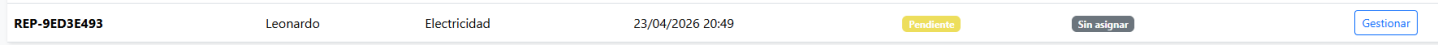
1. Modificamos modelo */Usuario.php* para añadir método para poder listar los usuarios

2. Actualizamos */DashboardController.php* para que muestre todos los usuarios para admins

3. Modificamos la vista */panel_admin.php* para añadir un modal con el form de crear aviso manual

4. Usaremos la misma ruta que usa el Particular (/nueva-incidencia), pero debemos modificar el controlador */IncidenciaController.php* para que sepa distinguir si lo envía un Cliente o un Admin para guardar el id correcto





→ Aviso creado manualmente para otro usuario desde el modal.

GESTIÓN DE INCIDENCIAS ADMIN

1. Modificamos modelo */Incidencia.php* para añadir métodos para listar técnicos, estados y actualizar incidencias.

2. Actualizamos */DashboardController.php* para que envíe técnicos y estados para admins

3. Actualizamos la vista */panel_admin.php*

2. Actualizamos */IncidenciaController.php* para atrapar este formulario y actualizar los datos

5. Añadimos la ruta a */index.php*

→ El admin puede gestionar incidencias

CALENDARIO

Usaremos FullCalendar de JS para el calendario, crearemos una pequeña API para no mezclar PHP y JS. Para esto, crearemos una URL que devuelva los datos en JSON desde la bbdd y así el calendario pueda leerlos.

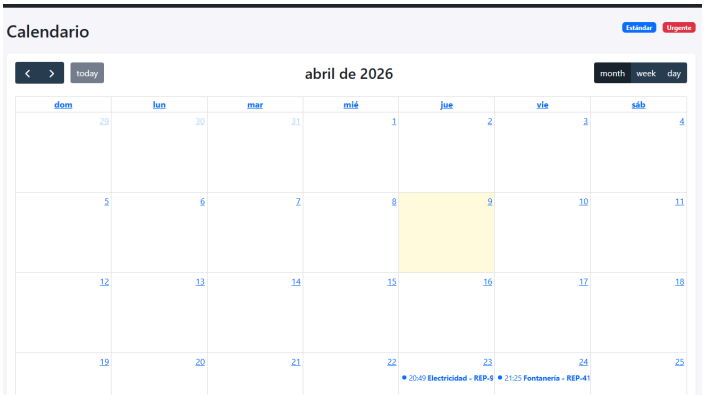
1. Modificamos */IncidenciaController.php* para añadir un método para traer los datos y devolver JSON

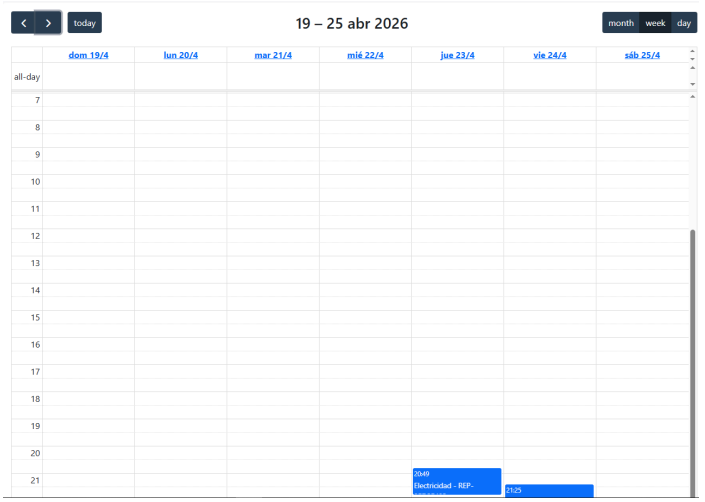
2. Actualizamos */DashboardController.php* para mostrar el calendario

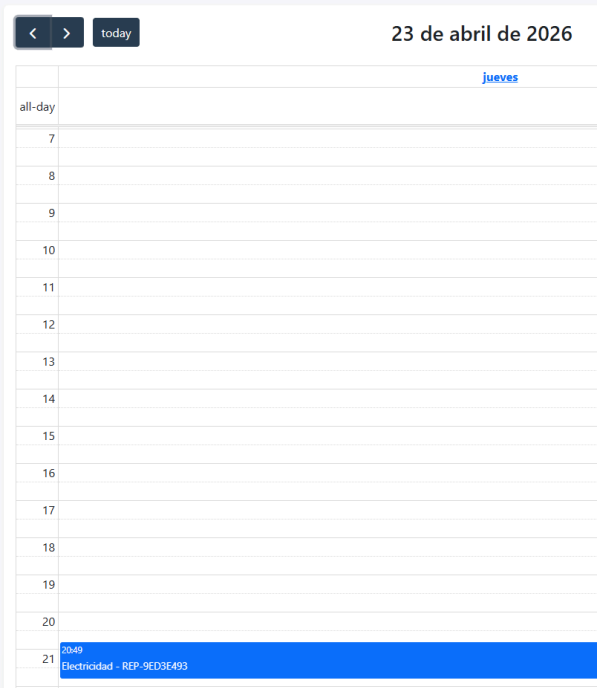
3. Añadimos las rutas a */index.php*

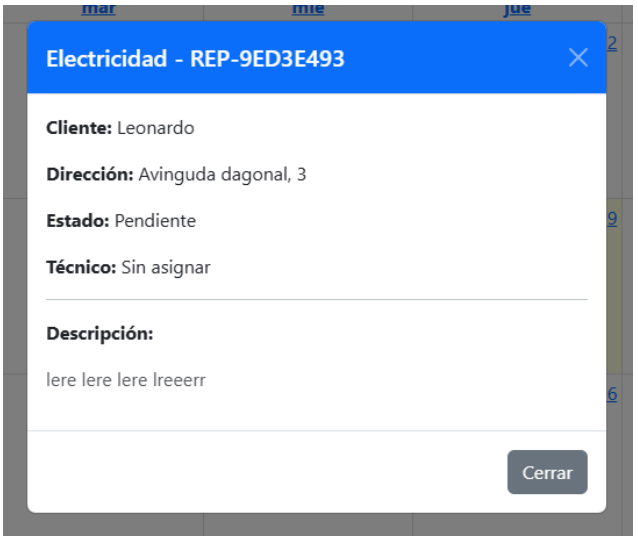
4. Creamos la vista */calendario_admin.php*

2. Actualizamos la vista */panel_admin.php* para añadir la ruta al botón









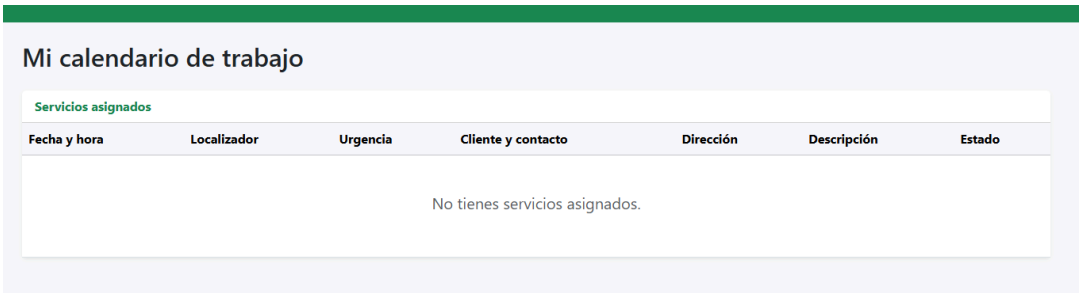
→ Calendario implementado con las vistas mes, semana y día y los eventos creados en display y vista de detalle.

PANEL TÉCNICO

1. Modificamos modelo */Incidencia.php* para añadir método para poder listar incidencias por técnico

2. Actualizamos */DashboardController.php* para que muestre todas las incidencias de este te´cnico

3. Creamos la vista */panel_tecnico.php*



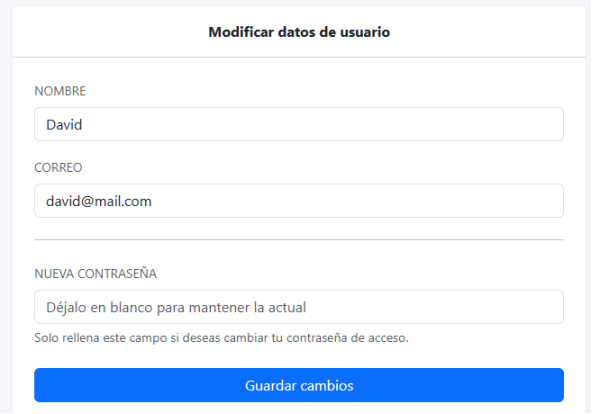
GESTIÓN DEL PERFIL

1. Modificamos modelo */Usuario.php* para que pueda actualizar los datos

2. Creamos el controlador */PerfilController.php* para mostrar el formulario y procesar los datos

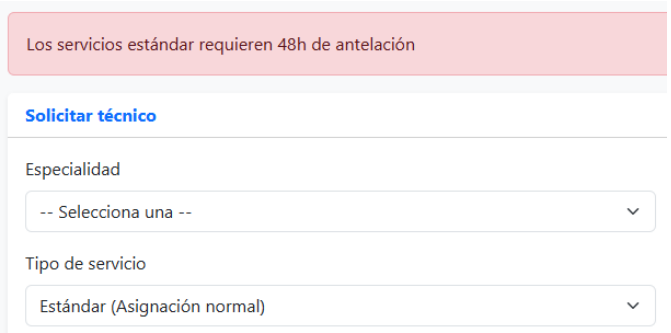
3. Creamos la vista */perfil.php*

4. Añadimos las rutas a */index.php*



RESTRICCIÓN 48H DESDE SERVIDOR

1. Modificamos el controlador */IncidenciaController.php* para calcular las 48h y rechazamos petición si no cumple



→ Petición rechazada al intentar fecha menor a 48h

CREACIÓN DE TÉCNICOS

Corrección del sistema de técnicos, se ha creado como tipo de usuario pero no se gestiona desde la tabla 'tecnicos' de la bbdd ni se asigna especialidad, crearemos un gestor maestro de técnicos. La lógica es que un técnico no se puede dar de alta a sí mismo, si no que tiene que venir desde un administrador, que podrá gestionar si está disponible y asignar especialidad.

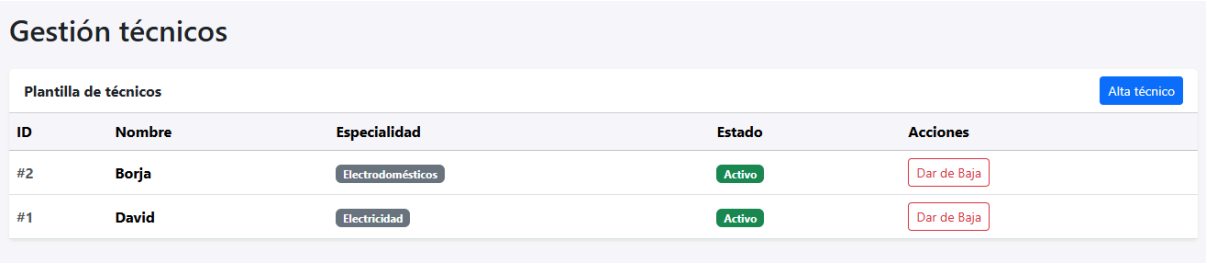
1. Modificamos el modelo */Usuario.php* para añadir un método para crear un técnico por admin y otro para poder cambiar su estado

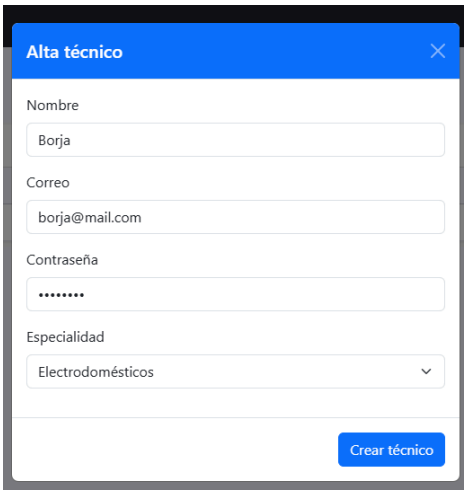
2. Creamos un controlador */MaestrosController.php* para estas acciones para separar lógica

3. Creamos la vista */panel_maestros.php*

4. Añadimos rutas a */index.php*

5. Añadimos botón a la vista */panel_admin.php*





→ Alta manual de técnico

8 Borja borja@mail.com $2y$10$kqmdyyFap4zYQa9KbfxN4uSbRiWRWS0UUC69TfyjVDY... tecnico → Tabla users

2 8 Borja 4 1 → Tabla técnicos

Estado	Acciones
Baja	Dar de Alta
Activo	Dar de Baja

→ Cambio de estado alta/baja

MEJORAS DE CRUD Y ACCIONES QUE FALTAN

CANCELAR AVISO (CLIENTE)

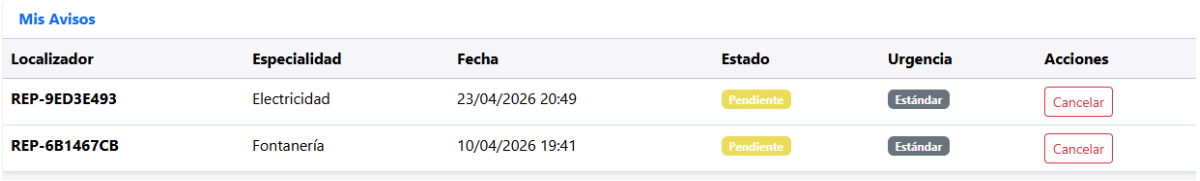
Cancelar aviso con restricción 48h

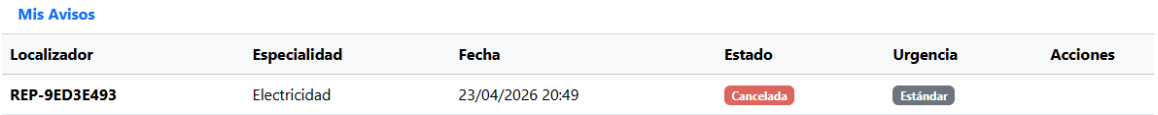
1. Modificamos el modelo */Incidencia.php* para que encuentre un aviso concreto y pueda cancelarlo

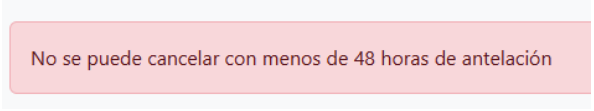
2. Modificamos el controlador */IncidenciaController.php* para que pueda cancelar

3. Añadimos el botón a la vista */panel_cliente.php*

4. Añadimos la ruta a */index.php*







→ Gestión de cancelación y restricción correcta.

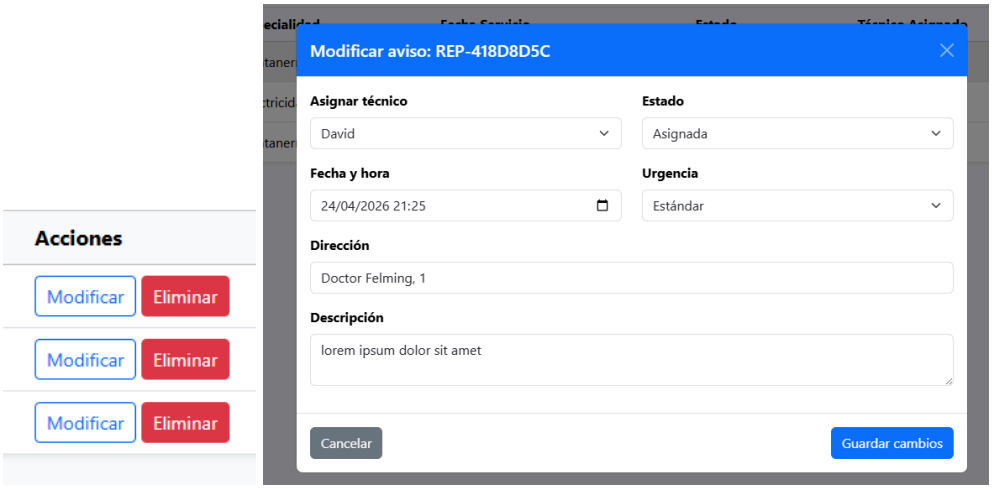
CANCELAR Y MODIFICAR AVISO (ADMIN)

1. Modificamos el modelo */Incidencia.php* para que pueda modificar todos los datos y añadimos opción eliminar

2. Modificamos el controlador */IncidenciaController.php* para que capte los nuevos datos editados y la eliminación

3. Añadimos el botón y los campos del formulario en la vista */panel_admin.php*

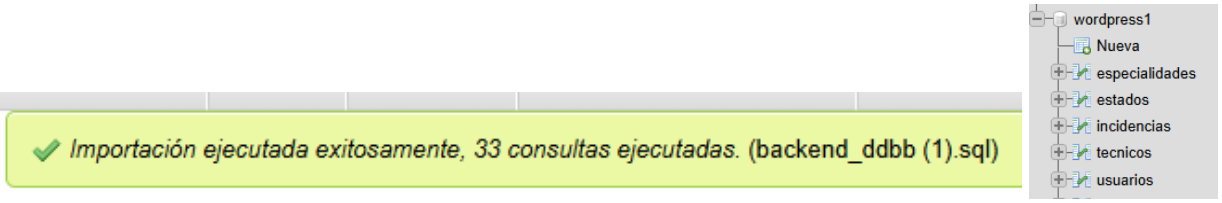
4. Añadimos la ruta a */index.php*



DESPLIEGUE A AWS

1. Exportamos la bbdd del phpMyAdmin local en un archivo sql.

2. Accedemos al phpMyAdmin del servidor e importamos la bbdd descargada.



3. Modificamos la configuración */database.php* para que apunte a la nueva bbdd

```
define('DB_HOST', 'localhost');
define('DB_NAME', 'wordpress1');
define('DB_USER', 'wordpress1');
define('DB_PASS', 'DWD8Ds3l4dvXpjZH');
```

4. Modificamos las vistas para que no se rompan, ya que estarán en una subcarpeta y no en un dominio.
<base href="/~uocx1/"> → En el header después del charset

5. Nos conectamos al servidor desde FileZilla

6. Copiamos la carpeta /src local a la carpeta */public_html* del servidor y los archivos de la carpeta */public* de local

7. Modificamos las rutas de navegación a src de */index.php* por la nueva ubicación en el servidor

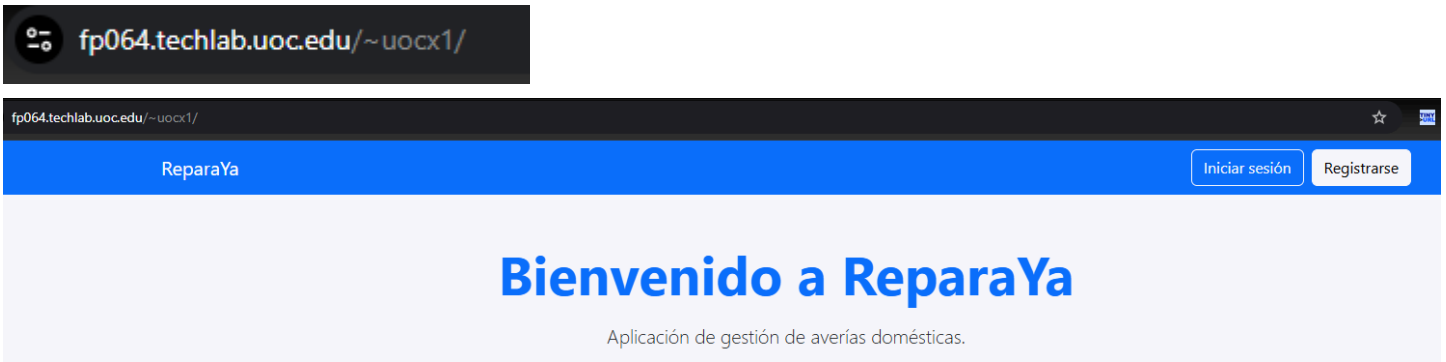
8. Modificamos url en */Router.php* para ignorar ruta de la uoc del servidor y cambiar nueva ruta de */src*

9. Modificamos los controladores en la función *render()* para la nueva ruta de */src,* y en todas las funciones con *require_once*, además de las redirecciones en headers.

10. Modificamos los *href* y *action* de las vistas para quitar la '/' para convertir url en relativas y que no ignore el *base*.

11. Modificamos el */.htaccess* para decirle que trabajamos en la carpeta de la uocx1.

8. Comprobamos subida de la aplicación al servidor



Aplicación subida al servidor.